企業級金流架構深度解析
融合 261016 與 261019最新架構
從邊界防禦到精準庫存，打造零缺陷的分散式支付防線
分散式賬本同步
智能合約安全認證
即時流動性管理
Gemini Notebook

分散式交易系統的混沌

狀態不一致：各系統狀態各自為政，導致掉單與重複扣款。

超賣危機：缺乏預留機制，併發請求擊穿庫存防線。

邊界模糊：商業邏輯與外部網路通訊高度耦合。

平台秩序重建

Clean Layered Architecture：嚴格依賴反轉，隔絕外部雜訊。

單向資料流與下游優先寫入：徹底消滅分散式狀態不一致。

三階段庫存狀態機：從預留到釋放的精準物理閥門。

Gemini Notebook

三方信任拓撲與資料權威定義

E_commerce (商城端)

訂單與庫存的唯一權威

掌管 Order、Payment、InventoryReservation。面向買家，發起交易與接收結果。

ecpay_mock (模擬金流)

金流狀態防線

掌管 gateway.Order、PaymentRequest。負責驗簽、轉送與回呼。

Bank_site (模擬銀行)

虛擬帳號的終極源頭

掌管 VirtualAccount。負責產生 bank_code、v_account 與 expire_date。

E_commerce (商城端)

訂單與庫存的唯一權威

掌管 Order、Payment、InventoryReservation。面向買家，發起交易與接收結果。

ecpay_mock (模擬金流)

金流狀態防線

掌管 gateway.Order、PaymentRequest。負責驗簽、轉送與回呼。

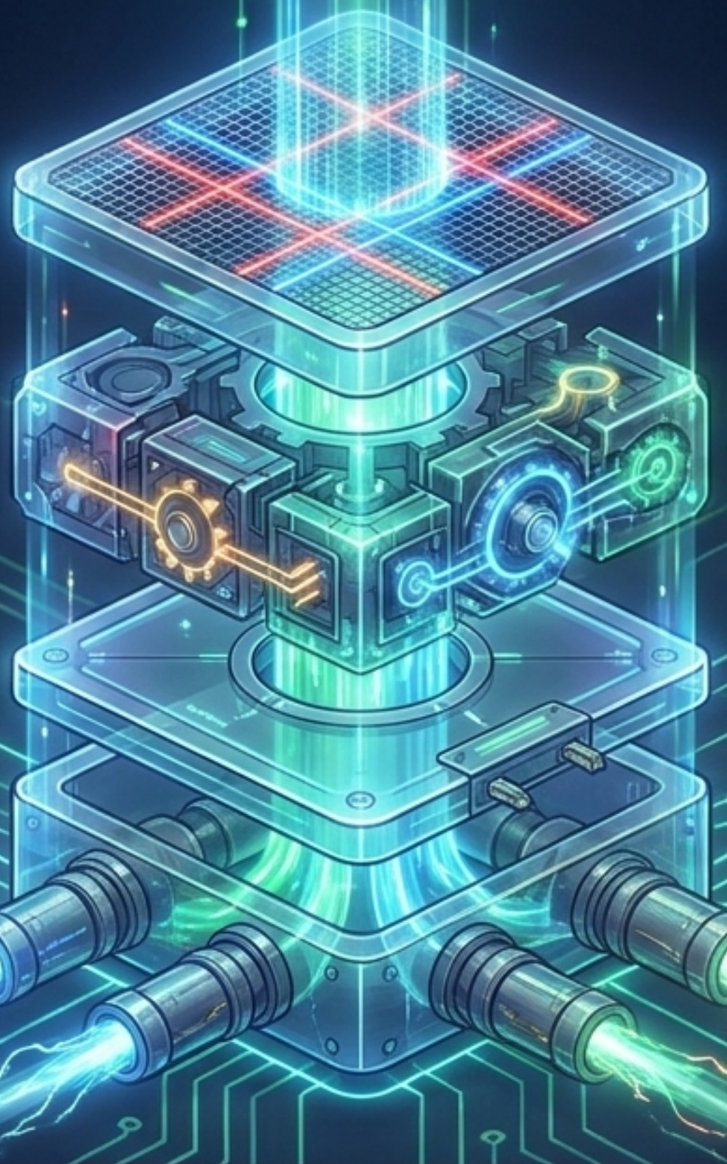
Bank_site (模擬銀行)

虛擬帳號的終極源頭

掌管 VirtualAccount。負責產生 bank_code、v_account 與 expire_date。

Gemini Notebook

Clean Architecture：隔離業務與底層的完美堆疊



Layer 1: View (過濾閘門)

職責：HTTP 邊界、路由解析、攔截並驗證 CheckMacValue。
限制：絕對不直接操作 Django ORM 或執行外部 HTTPX 呼叫。

Layer 2: Service (核心引擎)

職責：執行純粹的商業邏輯、狀態檢查、資料一致性控制。

Layer 3: Interface (抽象介面)

職責：定義跨層呼叫契約，實現依賴反轉，讓 Service 完全不知道外部實作細節。

Layer 4: Adapter (外部管線)

職責：處理 HTTPX 外部通訊、Timeout 控制、重新計算外部所需 CheckMacValue。

Gemini Notebook

雙重護城河模型：信任不跨越邊界的獨立簽章防護

架構哲學：簽章的生命週期只存在於兩個點對點的 Adapter 之間。絕對不將舊簽章偷渡至下游。

Gemini Notebook

分散式寫入最高原則：下游優先與非同步確認

核心價值：徹底避免「本地標記已付款，但下游系統沒收到」的致命災難。

1. 發起請求 (Initiate)
攔截狀態變更指令（如 Bank_site 模擬入帳）。

2. 預先通知 (Notify Downstream First)
透過 Adapter 向下游發送 HTTPX 通知。此時本地 DB 完全不動作。

3. 確認回應 (Await 200 OK)
嚴格檢查下游回應狀態。若失敗，直接拋出 Exception 中斷流程。

4. 本地寫入 (Write Local DB Last)
下游成功處理後，才開啟 transaction.atomic()，鎖定本地並更新狀態。

1. 發起請求 (Initiate)
攔截狀態變更指令 (如 Bank_site 模擬入帳)。

2. 預先通知 (Notify Downstream First)
透過 Adapter 向下游發送 HTTPX 通知。此時本地 DB 完全不動作。

3. 確認回應 (Await 200 OK)
嚴格檢查下游回應狀態。若失敗，直接拋出 Exception 中斷流程。

4. 本地寫入 (Write Local DB Last)
下游成功處理後，才開啟 transaction.atomic()，鎖定本地並更新狀態。

Gemini Notebook

庫存三階段物理閥門：避免超賣的狀態機

Gemini Notebook

雙向狀態瀑布：付款的逆流與取消的順流

Gemini Notebook

COD 懸浮沙盒：三階段 Session 落地保護機制

Stage 1 : 虛擬選擇 (select_store)
進入模擬綠界地圖選店。資料僅保留在瀏覽器與前端傳輸中。

Stage 2 : 懸浮沙盒 (review_store)
確認門市資料。資料暫存於 Session 沙盒，絕對不建立任何未確認的 ORM 記錄。

Stage 3 : 成功落地 (confirm_store)
點擊送出。呼叫下游商城端確認成功後，沙盒資料才真正落地寫入資料庫防堵惡意重放與髒資料。

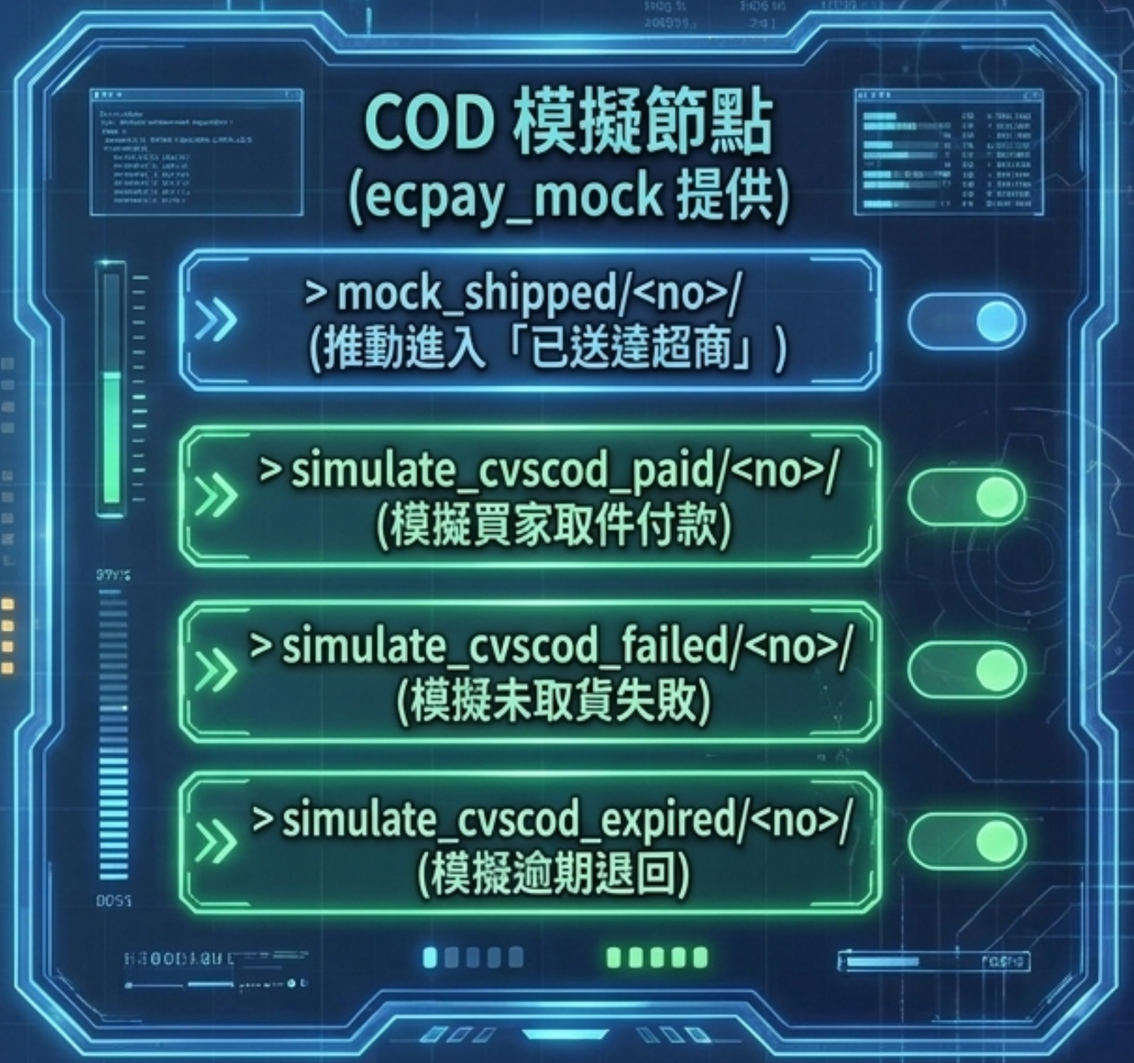
Gemini Notebook

架構基因解碼：ATM 與 COD 系統對比矩陣

診斷維度	ATM 虛擬帳號流程	COD 超商取貨付款
參與節點	(E_commerce ↔ ecpay_mock ↔ Bank_site)	(E_commerce ↔ ecpay_mock)
庫存 Commit 時機	收到 Bank_site 入帳回呼 (simulate_paid)	收到門市取件付款回呼 (simulate_cvscod_paid)
取消鏈路 (Cancel Chain)	貫穿三層：E -> P -> B (Bank 為鏈尾)	只到金流端：E -> P (ecpay_mock 為鏈尾)
終態保護限制	付款成功 (paid) 或逾期 (expired) 絕對鎖死	配送中 (cod_shipped) 後即不可取消
狀態起點	pending_payment (商城等待金流)	pending_payment → shipped (等待物流)

Gemini Notebook

後台中控模擬儀表板 (Admin Simulation Panel)



Gemini Notebook

終態守衛：絕對封鎖與條件式復活

防錯機制 1：Cancelled 鎖死防線

機制：在 Bank 模擬頁、P 通知層、E 端三層佈下嚴密守衛

守衛：if status == 'cancelled': raise ValueError

效果：訂單一旦進入取消終態，任何非法的後續 HTTP 回呼或人為模擬按鈕將被徹底無效化。

防錯機制 2：Expired 條件式復活 (特權金鑰)

機制：普通的逾期狀態為終態，系統全面拒絕修改。

特權：唯有透過後台模擬面板操作，Payload 中夾帶並簽入 MAC 的 simulate='1' 標記，才能啟動庫存重新預留機制讓訂單安全復活，便利 QA 測試。

Gemini Notebook

總結藍圖：堅不可摧的企業級支付防線

無縫的職責分離：從 View 到 Adapter 的潔淨架構，確保商業邏輯的純粹性。

無懈可擊的狀態機：先下游後本地的寫入原則與 Cancel Chain，讓資料永遠保持一致。

零信任的安全邊界：Dual CheckMacValue 保證跨系統通訊的絕對真確性。

企業級容錯力 (Enterprise-Grade Fault Tolerance)：不僅是模擬金流，更是強健的架構典範。

Gemini Notebook